

SICRES

Guide de l'équipe, semaine par semaine

Ce document explique, semaine par semaine, ce qu'il y a à faire pour construire le MVP de SICRES. Il est écrit pour que tout le monde dans l'équipe puisse comprendre — même sans avoir déjà travaillé sur ce type de projet. Chaque tâche est marquée d'un badge coloré pour indiquer si elle concerne le **frontend**, le **backend**, la **configuration/infra**, ou toute l'**équipe**. Pas besoin de tout lire d'un coup — reviens sur ce document au fur et à mesure, semaine après semaine.

Légende des badges :

BACKEND

FRONTEND

INFRA / CONFIG

ÉQUIPE

Deux réflexes à garder toute l'année : on avance une tâche à la fois, on commit régulièrement (petits commits, messages clairs), et on demande une relecture avant de merger sur la branche principale. Si un mot ou une consigne n'est pas clair : on demande, on ne devine pas.

Semaine 1 — Préparer le terrain

L'idée de la semaine : avant d'écrire le moindre code métier, on installe tout ce qu'il faut pour que le projet tourne sur la machine de chacun, de la même façon pour tout le monde.

INFRA / CONFIG

■ Nettoyer le dépôt : supprimer les vieux dossiers **Tikers/** et vérifier qu'il ne reste plus de trace de **sirfrontend/** (déjà fait — bien joué)

INFRA / CONFIG

■ Faire fonctionner **docker compose up** pour lancer les 4 services : le backend PHP, le frontend, la base de données PostgreSQL, et Mailpit (pour tester les emails sans en envoyer de vrais)

BACKEND

■ Vérifier que l'image PHP (**docker/php/Dockerfile.dev**) contient bien tout ce dont Laravel a besoin pour parler à PostgreSQL

BACKEND

■ Configurer le fichier **backend/.env.example** : c'est le modèle que tout le monde copie pour créer son propre .env (jamais le vrai .env ne doit être envoyé sur Git — il contient des mots de passe)

INFRA / CONFIG

■ Écrire quelques commandes simples dans le **Makefile** (make up, make migrate...) pour ne pas avoir à retaper des commandes Docker à rallonge

■ Pour comprendre : c'est quoi Docker ?

Docker permet de « mettre le projet dans une boîte » avec tout ce dont il a besoin pour tourner (PHP, la bonne version, les bonnes extensions...). Comme ça, si ça marche sur ta machine, ça marche pareil sur celle de ton collègue, et sur le serveur final. Sans Docker, chacun installerait PHP à sa façon et on aurait des bugs du style « ça marche chez moi mais pas chez toi ».

Le fichier .env contient les réglages propres à chaque machine (mot de passe de la base de données, adresse du serveur mail...). On ne le met jamais sur Git parce qu'il contient des informations sensibles — seul .env.example (sans les vraies valeurs) est partagé.

Semaine 2 — Les comptes et la connexion

L'idée de la semaine : avant de pouvoir faire quoi que ce soit dans l'application, il faut pouvoir se connecter. Et il faut que le système distingue clairement deux types de comptes : l'administrateur de la commune, et le responsable d'un établissement.

BACKEND

■ Créer la table **users** (déjà fournie par Laravel, à adapter) et deux tables séparées : **admin_communal** et **responsable_etablissement**

BACKEND

■ S'assurer qu'un compte ne peut JAMAIS être les deux en même temps (c'est une règle métier importante, décidée avec la commune)

BACKEND

■ Mettre en place la connexion (login) et la déconnexion avec **Laravel Sanctum** — `backend/app/Http/Controllers/Auth/`

BACKEND

■ Ajouter une limite de tentatives de connexion (pour éviter qu'on essaie 1000 mots de passe d'affilée)

FRONTEND

■ Page de connexion (**frontend/features/auth/**) + un endroit central qui garde en mémoire « qui est connecté » (**frontend/contexts/**)

BACKEND

■ Autoriser le frontend à parler au backend sans être bloqué par le navigateur (configuration CORS)

■ Pour comprendre : migration, mot de passe haché, token

Une migration, c'est un fichier qui décrit comment créer ou modifier une table dans la base de données, écrit en PHP plutôt qu'en SQL directement. L'avantage : tout le monde dans l'équipe applique les mêmes changements de base de données, dans le même ordre, juste en lançant `php artisan migrate`.

On ne stocke jamais un mot de passe en clair dans la base de données. On stocke un « hash » — une empreinte à sens unique, impossible à retransformer en mot de passe d'origine. Même si quelqu'un vole la base de données, il ne peut pas lire les mots de passe.

Un token, c'est un peu comme un ticket de vestiaire : après la connexion, le serveur donne un ticket au navigateur. Le navigateur le présente à chaque requête pour prouver « c'est bien moi, je suis déjà connecté » — sans avoir à retaper le mot de passe à chaque clic.

Semaine 3-4 — Les établissements

L'idée de la semaine : c'est le cœur du projet. Sans fiche établissement fiable, rien d'autre n'a de sens.

BACKEND

■ Créer la table **etablissements** avec tous les champs du cahier des charges (code, type, niveau, directeur, etc.)

BACKEND

■ Créer le modèle Eloquent correspondant (**backend/app/Models/**)

BACKEND

■ Construire les routes de l'API : lister, créer, modifier, consulter un établissement (**backend/app/Modules/Etablissements/**)

BACKEND

■ Ajouter une validation claire des champs obligatoires (pas de formulaire « à moitié rempli » accepté en base)

FRONTEND

■ Page de liste (avec recherche) et formulaire de création/modification (**frontend/features/etablissements/**)

FRONTEND

■ En parallèle : commencer une petite bibliothèque de composants réutilisables (boutons, champs de formulaire...) dans **frontend/components/ui/** — ça évitera de tout refaire à chaque nouvelle page

BACKEND

- Permettre à l'admin de créer un compte « responsable d'établissement » rattaché à une fiche établissement

■ Pour comprendre : API, endpoint, CRUD

Une API, c'est la façon dont le frontend (ce que voit l'utilisateur) et le backend (la base de données et la logique métier) se parlent. Concrètement, le frontend envoie une requête à une adresse précise (un endpoint, par exemple `/api/etablissements`), et le backend répond avec des données ou confirme une action.

CRUD est un mot qu'on utilise tout le temps : Create (créer), Read (lire), Update (modifier), Delete (supprimer). Presque tous les modules de ce projet sont, au fond, un CRUD avec des règles en plus.

Semaine 4-5 — Les campagnes de recensement

L'idée de la semaine : avant qu'un établissement puisse déclarer quoi que ce soit, il faut qu'une campagne soit ouverte. C'est la commune qui décide quand.

BACKEND

- Créer la table **campagnes** (titre, année, dates, statut)

BACKEND

- API : création d'une campagne, bouton « ouvrir » / « clôturer » (**backend/app/Modules/Campagnes/**)

BACKEND

- Vérifier que seul un admin communal peut ouvrir/clôturer une campagne — pas un responsable d'établissement

FRONTEND

- Liste des campagnes + actions visibles seulement pour l'admin (**frontend/features/campagnes/**)

■ Pour comprendre : pourquoi vérifier les droits à deux endroits (backend ET frontend) ?

Sur le frontend, on cache les boutons qu'un responsable d'établissement ne devrait pas voir — c'est du confort d'utilisation. Mais ça ne suffit pas : quelqu'un d'un peu curieux pourrait appeler l'API directement sans passer par les boutons. C'est pour ça qu'on vérifie aussi les droits côté backend — c'est la vraie protection. Retiens cette règle : le frontend améliore l'expérience, le backend protège les données.

Semaine 5-6 — Les déclarations

L'idée de la semaine : c'est la fonctionnalité la plus attendue par les établissements — pouvoir remplir et envoyer leurs chiffres pour une campagne donnée.

BACKEND

- Créer la table **declarations** (effectifs filles/garçons, nombre de classes, de salles, besoins prioritaires...)

BACKEND

- Ajouter une règle très importante en base de données : un établissement ne peut pas soumettre deux déclarations pour la même campagne

BACKEND

- API : sauvegarde en brouillon (on peut revenir dessus), puis soumission définitive (**backend/app/Modules/Declarations/**)

BACKEND

- Un responsable ne doit voir/modifier que les déclarations de SON établissement (jamais celles d'un autre)

FRONTEND

- Formulaire avec sauvegarde de brouillon + écran de confirmation avant la soumission finale (**frontend/features/declarations/**)

BACKEND

- Commencer à écrire des tests automatiques simples sur ce module (voir encadré semaine 10)

■ Pour comprendre : une contrainte « UNIQUE » en base de données

Dire à la base de données « la paire (établissement, campagne) doit être unique dans la table `declarations` », c'est lui demander de refuser elle-même toute tentative de doublon — même si un bug dans le code essaie d'en créer un. C'est une sécurité de dernier recours : on ne compte pas uniquement sur le code applicatif pour éviter les erreurs, on verrouille aussi au niveau de la base.

Semaine 6-7 — Les pièces justificatives

L'idée de la semaine : les établissements doivent pouvoir prouver ce qu'ils déclarent, en joignant des documents.

BACKEND

■ Créer la table **`pieces_justificatives`** (type de document, nom du fichier, statut de validation)

BACKEND

■ API d'upload : recevoir un fichier, le stocker au bon endroit sur le serveur, l'associer à une déclaration (**`backend/app/Modules/Documents/`**)

BACKEND

■ Limiter les types de fichiers acceptés (PDF, JPG, PNG) et la taille maximale

FRONTEND

■ Bouton d'upload + liste des documents déjà envoyés (**`frontend/features/documents/`**)

■ Pour comprendre : où vont les fichiers uploadés ?

Quand quelqu'un envoie un fichier, on ne le met surtout pas dans le dossier `public/` du serveur (n'importe qui pourrait alors deviner l'adresse et le télécharger sans autorisation). On le range dans un espace de stockage à part, et c'est le backend qui vérifie — à chaque téléchargement — que la personne qui le demande a bien le droit de le voir.

Semaine 7 — La validation

L'idée de la semaine : l'admin communal doit pouvoir accepter ou refuser une déclaration, avec une justification en cas de refus.

BACKEND

■ Créer la table **`validations`** (décision, motif, qui a validé, quand)

BACKEND

■ API : bouton « valider » / « rejeter » — le motif est obligatoire en cas de rejet (**`backend/app/Modules/`**, nouveau dossier `Validations`)

BACKEND

■ Quand une validation est enregistrée, le statut de la déclaration doit changer automatiquement — jamais l'inverse (on ne modifie jamais le statut « à la main » sans passer par une validation)

FRONTEND

■ Liste des déclarations en attente + actions valider/rejeter (**`frontend/features/declarations/`**, vue admin)

■ Pour comprendre : c'est quoi une « policy » ?

Une policy (en français, une règle d'autorisation), c'est une classe PHP dédiée qui répond à une seule question : « cette personne a-t-elle le droit de faire cette action sur cette donnée précise ? ». Plutôt que d'écrire des `if ($user->role === 'admin')` un peu partout dans le code (et d'en oublier certains), on centralise cette logique à un seul endroit par type de donnée. Ici : seul un admin communal peut valider ou rejeter une déclaration, jamais un responsable d'établissement.

Semaine 8 — Le tableau de bord

L'idée de la semaine : rendre visible tout le travail de recensement déjà fait — un admin doit pouvoir voir l'état d'avancement en un coup d'œil.

BACKEND

■ API qui calcule les totaux (nombre d'établissements par type, par niveau, taux de déclarations soumises) — **backend/app/Modules/Dashboard/**

BACKEND

■ Attention : préparer toutes les données en un seul appel API, pas dix appels séparés (voir encadré)

BACKEND

■ Créer quelques données de test réalistes (seeder) pour avoir quelque chose à afficher pendant qu'on développe

FRONTEND

■ Page d'accueil avec les chiffres clés et un ou deux graphiques simples (**frontend/features/dashboard/**)

■ Pour comprendre : le problème du « N+1 »

Imagine une liste de 50 établissements, et pour chacun, on va chercher sa commune dans une requête séparée : ça fait 51 requêtes à la base de données au lieu d'une ou deux. C'est ce qu'on appelle le problème « N+1 », et c'est une des causes les plus fréquentes de lenteur dans une application. La solution s'appelle l'« eager loading » (chargement anticipé) : on dit à Laravel à l'avance « quand tu vas chercher les établissements, ramène aussi leur commune direct » — une seule requête bien construite au lieu de 51.

Semaine 9 — Les exports (PDF / Excel)

L'idée de la semaine : permettre à la commune de récupérer les données pour ses propres besoins (réunions, transmission à la DRENA...).

BACKEND

■ Service backend qui génère un export PDF et un export Excel à partir des données d'une campagne (**backend/app/Services/**)

FRONTEND

■ Bouton d'export côté frontend, avec un nom de fichier clair (commune + campagne + date)

FRONTEND

■ Vérifier que l'Excel généré s'ouvre bien sans erreur (teste-le toi-même dans Excel ou LibreOffice avant de dire que c'est fini)

BACKEND

■ Mettre à jour la documentation de l'API (**docs/06-API.md**) pour que ce qu'on a construit depuis la semaine 2 soit noté quelque part

Semaine 10 — La recette : on teste pour de vrai

L'idée de la semaine : jusqu'ici on a testé entre nous. Cette semaine, on regarde si un vrai administrateur communal et de vrais établissements arrivent à utiliser l'outil sans notre aide.

BACKEND

■ Finir d'écrire les tests automatiques du backend pour les modules principaux (**backend/tests/Feature/**) : au minimum, un cas qui marche + un cas qui doit être refusé (ex. deux déclarations pour la même campagne)

FRONTEND

■ Vérifier qu'il n'y a aucune erreur ESLint/TypeScript côté frontend

ÉQUIPE

■ Organiser une session avec un vrai administrateur de Port-Bouët (et si possible 2-3 responsables d'établissement), sur un environnement de test — pas la vraie production

ÉQUIPE

- Noter chaque blocage ou incompréhension rencontré pendant cette session, même les petits détails

■ Pour comprendre : test automatique vs recette

Un test automatique, c'est du code qui vérifie du code — utile pour attraper vite un bug technique (ex. « la contrainte unique fonctionne-t-elle vraiment ? »). Une recette, c'est un humain qui utilise l'application comme un vrai utilisateur le ferait, sans connaître les coulisses. Les deux sont indispensables et ne remplacent pas l'un l'autre : un test automatique ne dira jamais « ce bouton n'est pas assez visible » ou « je ne comprends pas ce message d'erreur » — seule une vraie personne le remarquera.

Semaine 11 — On solidifie avant le grand jour

L'idée de la semaine : corriger ce qui a été trouvé en semaine 10, et s'assurer que les bases de sécurité sont bien en place avant l'ouverture au public.

ÉQUIPE

- Corriger en priorité tout ce qui empêchait un testeur de terminer son parcours pendant la recette

INFRA / CONFIG

- Mettre en place le certificat HTTPS (accès sécurisé au site)

INFRA / CONFIG

- Activer la sauvegarde automatique quotidienne de la base de données, et la laisser tourner au moins 3 jours pour vérifier qu'elle fonctionne vraiment

INFRA / CONFIG

- Tester une restauration de sauvegarde au moins une fois (pas juste la créer — vérifier qu'on peut vraiment s'en servir en cas de problème)

■ Pour comprendre : HTTPS et sauvegardes, pourquoi c'est non négociable

HTTPS, c'est le petit cadenas dans la barre d'adresse du navigateur : il chiffre ce qui circule entre le navigateur et le serveur, pour qu'un mot de passe ou une donnée d'établissement ne puisse pas être lu en clair par quelqu'un sur le même réseau. Pour une administration qui manipule des données sur des enfants et des établissements, ce n'est pas une option.

Une sauvegarde, c'est une copie de la base de données prise régulièrement, gardée à part. Si le serveur tombe en panne ou si quelqu'un supprime une donnée par erreur, la sauvegarde permet de tout récupérer. Une sauvegarde qu'on n'a jamais essayé de restaurer n'est qu'une sauvegarde théorique — d'où le test de restauration.

Semaine 12 — Le lancement

L'idée de la semaine : on passe du test contrôlé à l'usage réel.

ÉQUIPE

- Vérifier une dernière checklist avant l'ouverture (HTTPS actif, sauvegardes actives, aucun compte de test qui traîne en production)

ÉQUIPE

- L'admin communal ouvre officiellement la première campagne réelle

ÉQUIPE

- Communiquer aux établissements de la commune que la plateforme est ouverte

ÉQUIPE

- Surveiller le tableau de bord dans les jours qui suivent pour voir si les établissements arrivent bien à se connecter et déclarer

■ Pour comprendre : c'est quoi un « déploiement » ?

Déployer, c'est mettre la dernière version du code sur le serveur que tout le monde utilise (par opposition à ta machine, où toi seul travailles). Idéalement, ça se fait par un processus automatisé et répétable (un « pipeline de déploiement ») plutôt qu'à la main — pour être sûr de ne rien oublier et pouvoir refaire l'opération de la même façon la prochaine fois.

Une dernière chose

Ce document donne le fil de l'eau, semaine par semaine, mais la réalité d'un projet, c'est que certaines semaines vont déborder sur la suivante, et ce n'est pas grave. Le plus important : si tu bloques plus de quelques heures sur quelque chose, préviens l'équipe plutôt que de rester seul dessus. Un blocage partagé se résout plus vite qu'un blocage silencieux.